Projeto e Análise de Algoritmos — Prova I de 2025

Resolução comentada · IFMG, Campus Formiga · Bacharelado em Ciência da Computação
Professor: Mário Luiz Rodrigues Oliveira · 25 pontos · 120 minutos

---

> **Por que esta prova importa.** Cinco das onze questões saem direto das listas de exercícios — uma delas idêntica palavra por palavra (2(c)) e outra a menos de uma expressão (4(a)). As tarjas DE ONDE VEIO indicam a origem de cada uma. **Atenção à Questão 2(a):** ela é a Q3(c) da Lista 2 com o sinal trocado de $+$ para $-$, e a resposta inverte de verdadeira para falsa. Decorar o gabarito da lista faz perder os pontos.

Cabeçalho, código de honra e instruções da prova (transcrição literal)

Instituto Federal de Educação, Ciência e Tecnologia · Curso: Bacharelado em Ciência da Computação · Disciplina: Projeto Análise de Algoritmos · Professor: Mário Luiz Rodrigues Oliveira · Atividade: Prova I · Formiga, MG, 13 de junho de 2025

Código de honra e conduta discente:
Eu, _____, matrícula _____ prometo pela minha honra que fui honesto(a) e não trapaceei nessa avaliação passando ou recebendo cola.
Formiga, MG, 13 de junho de 2025.

Instruções:

1. Esta atividade deve, obrigatoriamente, ser resolvida **individualmente**.
2. Caso você ache que falta algum detalhe nas especificações, você deverá fazer as suposições que julgar necessárias e escrevê-las junto à resolução da atividade. Pode acontecer também que a descrição desta atividade contenha dados e/ou especificações supérfluas para sua solução. Utilize sua capacidade de julgamento para separar o supérfluo do necessário.
3. Na(s) folha(s) contendo a solução desta atividade coloque seu nome e matrícula. Na correção desta prova será considerado apenas e tão somente o conteúdo que estiver na(s) folha(s) de solução.
4. **Não basta apresentar apenas o(s) resultado(s). É necessário mostrar a resolução ou o raciocínio utilizado para responder às questões. Portanto, justifique formalmente todas as suas respostas. Questões sem respostas ou com respostas sem justificativas não serão consideradas**.
5. É permitido aos discentes consultar materiais impressos, tais como: livros, materiais disponibilizados pelo professor e anotações produzidas pelo discente.
6. Não são permitidas consultas a materiais/conteúdos disponíveis na *Web*.
7. Durante a realização desta atividade não é permitido o uso de celulares e outros dispositivos eletrônicos; bem como também conversas e/ou diálogos com colegas.
8. Seja honesto(a) e lembre-se: você deu sua palavra de honra.
9. **Plágio ou a tentativa de plágio implicará em nota 0 (zero) a todos os envolvidos**. Sem prejuízo de demais sanções/pebalidades cabíveis nesta situação.
10. O valor desta atividade é **25 pontos** e você terá até 120 minutos para resolvê-la.

Questão 1 (5,0 pontos)

Enunciado

Você está em um campo minado infinitamente grande. Você tem um detector de minas que só consegue identificar uma mina localizada no máximo a um passo de distância. Você deve encontrar a primeira mina do campo, mas a única informação que você tem sobre esta mina é que ela está diretamente a sua esquerda ou diretamente a sua direita. Entretanto, você não sabe a quantidade m de passos de onde você está até a mina.
(3.0 pontos) Projete um algoritmo para achar a mina em O($f(m)$) passos. A função $f(m)$ deve ser polinomial no parâmetro m.
(2.0 pontos) Argumente que seu algoritmo tem a complexidade requerida.

Resposta. Busca por duplicação (exponential search): sonda-se o detector na origem ANTES de tudo e, em seguida, alternando os dois lados, para i = 0, 1, 2, ... anda-se 2^i passos para a direita (checando o detector a CADA passo), volta-se à origem, anda-se 2^i passos para a esquerda (checando a cada passo) e volta-se à origem. O algoritmo encontra a mina gastando menos de 14m passos, isto é, O(m) passos — logo f(m) = m, que é polinomial em m.

PARTE 1 — O ALGORITMO (3,0)

Modelo. A origem é a posição 0; a mina está na posição +m ou -m, com m >= 1 inteiro desconhecido. O detector, estando o agente na posição p, acusa a mina se ela estiver a no máximo um passo de distância, ou seja, se |p - (posição da mina)| <= 1. Um "passo" é o movimento de uma unidade; contamos passos.

Por que não basta varrer um lado só. O campo é infinito e não se sabe o lado. Se o algoritmo caminhasse indefinidamente para a direita e a mina estivesse à esquerda, ele nunca terminaria. Portanto qualquer algoritmo correto tem de alternar os dois lados, e cada troca de lado obriga a voltar pela origem. É esse vaivém que precisa ser contabilizado.

Algoritmo (busca por duplicação, com sondagens alternadas):

```
se o detector acusar ja na origem, PARE (a mina esta a um passo)
  i <- 0
  repita
     ande 2^i passos para a DIREITA, acionando o detector a cada passo;
        se o detector acusar, PARE (mina encontrada)
     volte 2^i passos, retornando à origem
     ande 2^i passos para a ESQUERDA, acionando o detector a cada passo;
        se o detector acusar, PARE (mina encontrada)
     volte 2^i passos, retornando à origem
     i <- i + 1
  fim repita
```

Dois pontos essenciais. Primeiro, e preciso SONDAR A ORIGEM antes de entrar no laco: sem isso, com m = 1 o agente nao consulta o detector estando a um passo da mina e acaba PISANDO nela (para a mina em -1 ele gasta 3 passos e pisa; para +1, gasta 1 e pisa). Com a sondagem inicial, m = 1 e resolvido em 0 passos. Ressalva honesta: na origem o detector acusa se e so se m = 1, mas nao informa se a mina esta em +1 ou em -1. Nenhum algoritmo que nao pise na mina consegue distinguir os dois casos, entao 'encontrar' aqui significa detectar a presenca a um passo, que e o maximo que o detector permite. Segundo, o detector tem alcance 1, então NÃO se pode apenas testar no ponto final do salto de 2^i passos — a mina estaria em alguma posicao intermediaria do trajeto e o agente PISARIA nela antes de qualquer sondagem — o passo e o movimento de uma unidade, entao ele percorre fisicamente todas as posicoes do caminho. É preciso acionar o detector a cada passo do percurso. Isso não aumenta o número de passos (andar 2^i unidades custa 2^i passos, com ou sem sondagem), mas é o que garante a corretude e o que torna a contagem abaixo legítima.

Corretude e término. Seja k o menor inteiro k >= 0 com 2^k >= m. Ele existe porque as potências de 2 crescem sem limite. Na iteração i = k, ao varrer o lado correto o agente percorre todas as posições 1, 2, ..., 2^k desse lado; como m <= 2^k, o agente alcanca a posicao m-1 ainda nessa iteracao. Como o detector e acionado a cada passo e tem alcance 1, ele acusa a mina ali e o agente PARA — sem nunca ocupar a posicao m. (Simulando o pseudocodigo para m = 1 ate 2999 nos dois lados, o agente jamais pisa na mina: ele para em m-1, ou na propria origem quando m = 1.) Logo o algoritmo termina, e termina na iteração k ou antes.

PARTE 2 — COMPLEXIDADE (2,0)

Seja T(m) o número de passos no pior caso.

(i) Iterações completas 0, 1, ..., k-1. Como cota superior, suponha o pior: nenhuma delas encontra a mina (encontrar antes só diminui o custo). Uma iteração completa i custa 2^i (ida à direita) + 2^i (volta) + 2^i (ida à esquerda) + 2^i (volta) = 4·2^i passos. Somando a progressão geométrica em forma fechada: soma_{i=0}^{k-1} 4·2^i = 4·(2^k - 1).

(ii) Iteração k. Nela a mina é certamente encontrada. O pior caso é a mina estar à ESQUERDA, pois aí gasta-se a varredura completa à direita (2^k), a volta (2^k) e no máximo mais 2^k passos à esquerda até acusá-la: no máximo 3·2^k passos. (Se estivesse à direita, custaria no máximo 2^k, ainda menos.)

Total: T(m) <= 4·(2^k - 1) + 3·2^k = 7·2^k - 4 < 7·2^k.

Falta relacionar 2^k com m. Pela MINIMALIDADE de k: - se k >= 1, então 2^(k-1) < m, e multiplicando por 2: 2^k < 2m; - se k = 0, então 2^k = 1 <= m < 2m (pois m >= 1). Em ambos os casos 2^k < 2m. Portanto T(m) < 7·(2m) = 14m para todo m >= 1.

Testemunhas: c = 14 e m0 = 1 satisfazem 0 <= T(m) <= c·m para todo m >= m0. Conferindo em m = m0 = 1 exatamente: com a sondagem inicial, T(1) = 0 (a mina esta a um passo da origem e e acusada antes de qualquer movimento), e 0 <= 14*1. Sem a son-

dagem inicial o pior caso seria T(1) = 3 (mina em -1), e 3 <= 14*1 tambem vale. A desigualdade e satisfeita em m0 nos dois casos. Simulando m = 1 ate 299, a maior razao T(m)/m observada foi 12,888 (em m = 258), sempre abaixo de 14.

Conclusão: T(m) = O(m). Tomando f(m) = m, o algoritmo roda em O(f(m)) passos com f polinomial (grau 1) em m, como exigido.

Observação sobre alternativas. A estratégia ingênua de alternar os lados com incrementos 1, 2, 3, 4, ... (isto é, ir j passos para cada lado na j-ésima rodada) também é correta, mas custa 4*(1 + 2 + ... + m) = 2m(m+1) = Theta(m^2) passos — o fator 4 vem de ida e volta em cada um dos dois lados, e o somatorio 1+2+...+m sozinho contaria apenas as idas de um lado: é polinomial e portanto aceitável pelo enunciado, mas estritamente pior. A duplicação atinge Theta(m), que é ótimo a menos de constante, ja que qualquer algoritmo precisa de pelo menos m-1 passos para chegar a vizinhanca da mina — e isso se justifica assim: para acusar a mina, que esta a distancia m, o agente precisa ocupar alguma posicao a distancia no maximo 1 dela, e partindo da origem isso exige no minimo m-1 movimentos.

Questão 2(a) (2,0 pontos)

> Enunciado
> Prove que as afirmativas abaixo são verdadeiras ou falsas, usando para isso as definições das notações assintóticas e/ou contra exemplos. Considere que as funções $f(n)$ e $g(n)$ são assintoticamente não negativas.
> a) (2.0 pontos) Se $f(n) \in O(u(n))$ e $g(n) \in O(v(n))$ então $f(n) - g(n) \in O(u(n) - v(n))$.

DE ONDE VEIO Lista 2, Q3(c) — **mesma sentença com + trocado por −, e a resposta se inverte**

Resposta. FALSA.

AFIRMATIVA: se f(n) = O(u(n)) e g(n) = O(v(n)), então f(n) - g(n) = O(u(n) - v(n)).

É FALSA. Dois motivos independentes.

Motivo 1 — contraexemplo concreto. Tome f(n) = 2n, u(n) = n, g(n) = n, v(n) = n. Todas são assintoticamente não negativas.

(1) f(n) = O(u(n)): tomando c = 2 e n0 = 1, vale 0 <= 2n <= 2·n para todo n >= 1. Em n = n0 = 1: 0 <= 2 <= 2. OK. (2) g(n) = O(v(n)): tomando c = 1 e n0 = 1, vale 0 <= n <= 1·n para todo n >= 1. Em n = n0 = 1: 0 <= 1 <= 1. OK.

Mas f(n) - g(n) = 2n - n = n e u(n) - v(n) = n - n = 0. Se f - g estivesse em O(u - v), existiriam c > 0 e n0 tais que 0 <= n <= c·0 = 0 para todo n >= n0. Isso obriga n <= 0 para todo n >= n0, o que é falso já em n = n0 (por exemplo, para qualquer n0 >= 1, em n = n0 >= 1 teríamos 1 <= n0 <= 0, absurdo). Como nenhum par (c, n0) serve, f(n) - g(n) NÃO pertence a O(u(n) - v(n)).

Motivo 2 — a expressão pode nem estar bem definida. A definição de O(h(n)) pressupõe h assintoticamente não negativa (é a hipótese padrão do CLRS para que a classe faça sentido). Nada nas hipóteses garante que u(n) - v(n) seja assintoticamente não negativa: basta tomar u(n) = n e v(n) = n^2, com f(n) = n em O(u) e g(n) = n^2 em O(v); aí u(n) - v(n) = n - n^2 < 0 para todo n >= 2, e a própria notação O(u - v) fica mal formada. Ou seja, além de a implicação falhar, o lado direito pode sequer denotar uma classe legítima.

A raiz do erro: a notação O é fechada para SOMA (se f = O(u) e g = O(v), então f + g = O(u + v), pois as cotas superiores se somam), mas não para subtração: subtrair destrói a monotonia das desigualdades — de f <= c1·u e g <= c2·v não se conclui nada sobre f - g comparado a u - v, porque a cota de g é superior, e o que se precisaria ali seria uma cota INFERIOR de g.

Questão 2(b) (2,0 pontos)

> Enunciado
> Prove que as afirmativas abaixo são verdadeiras ou falsas, usando para isso as definições das notações assintóticas e/ou contra exemplos. Considere que as funções $f(n)$ e $g(n)$ são assintoticamente não negativas.
> b) (2.0 pontos) Se $f(n) \in O(g(n))$ então $g(n) \in \Omega(f(n))$.

DE ONDE VEIO Lista 2, Q3(a) — uma das duas direções do "se e somente se"

Resposta. VERDADEIRA.

AFIRMATIVA: se f(n) = O(g(n)), então g(n) = Omega(f(n)).

É VERDADEIRA. As duas notações expressam a MESMA desigualdade, lida de um lado e do outro.

Hipótese. f(n) = O(g(n)) significa: existem uma constante c > 0 e um n0 > 0 tais que 0 <= f(n) <= c·g(n) para todo n >= n0. (*) Aqui c e n0 são fixados de uma vez e não dependem de n.

Tese. Queremos exibir uma constante d > 0 e um m0 > 0 tais que 0 <= d·f(n) <= g(n) para todo n >= m0, que é exatamente a definição de g(n) = Omega(f(n)).

Construção das testemunhas. Tome d = 1/c e m0 = n0. A escolha é legítima: como c > 0 estritamente, 1/c está definido e 1/c > 0. Além disso d e m0 são constantes — não dependem de n.

Verificação. Seja n >= m0 = n0 qualquer. Por (*), f(n) <= c·g(n). Dividindo ambos os lados por c > 0 (o que preserva o sentido da desigualdade, pois c é positivo): (1/c)·f(n) <= g(n), isto é, d·f(n) <= g(n). Falta a não negatividade: por (*) também vale f(n) >= 0, e como d > 0, temos d·f(n) >= 0. Juntando, 0 <= d·f(n) <= g(n) para todo n >= m0.

Isso é precisamente a definição de g(n) = Omega(f(n)), com testemunhas d = 1/c e m0 = n0. Como o argumento vale para qualquer par (c, n0) dado pela hipótese, a implicação é verdadeira em geral.

Exemplo ilustrativo (não é a prova, apenas leitura concreta): f(n) = n, g(n) = n^2, com c = 1 e n0 = 1 dão n <= n^2 para n >= 1; então d = 1 e m0 = 1 dão 0 <= n <= n^2, ou seja, n^2 = Omega(n).

Questão 2(c) (2,0 pontos)

> Enunciado
> Prove que as afirmativas abaixo são verdadeiras ou falsas, usando para isso as definições das notações assintóticas e/ou contra exemplos. Considere que as funções $f(n)$ e $g(n)$ são assintoticamente não negativas.
> c) (2.0 pontos) Se $f(n) \in O(g(n))$ e $g(n) \in O(f(n))$ então $f(n) \in \theta(g(n))$.

de onde veio Lista 2, Q3(d) — **o item é idêntico, palavra por palavra** (o enunciado comum da lista diz "sentenças"; o da prova, "afirmativas")

Resposta. VERDADEIRA (é o Teorema 3.1 do CLRS, depois de reescrever g = O(f) como f = Omega(g) usando o item (b) com os papéis trocados).

AFIRMATIVA: se f(n) = O(g(n)) e g(n) = O(f(n)), então f(n) = Theta(g(n)).

É VERDADEIRA.

Hipóteses (com nomes de constantes distintos, para não colidirem). (H1) f(n) = O(g(n)): existem b > 0 e n1 > 0 com 0 <= f(n) <= b·g(n) para todo n >= n1. (H2) g(n) = O(f(n)): existem a > 0 e n2 > 0 com 0 <= g(n) <= a·f(n) para todo n >= n2.

De (H2), dividindo por a > 0 (permitido, pois a é estritamente positivo, e a divisão por positivo preserva a desigualdade): (1/a)·g(n) <= f(n) para todo n >= n2. (**) Ou seja, (H2) fornece uma cota INFERIOR para f em termos de g — que é justamente o que falta para Theta.

Tese. Queremos c1 > 0, c2 > 0 e n0 > 0 com 0 <= c1·g(n) <= f(n) <= c2·g(n) para todo n >= n0, que é a definição de f(n) = Theta(g(n)).

Construção das testemunhas. Tome c1 = 1/a, c2 = b, n0 = max(n1, n2). São legítimas: a > 0 garante c1 = 1/a > 0; b > 0 por (H1); e n0 é uma constante, o máximo de duas constantes, não dependendo de n.

Verificação. Seja n >= n0 qualquer. Então n >= n1 e n >= n2 simultaneamente, de modo que valem (H1) e (**) ao mesmo tempo: - de (**): c1·g(n) = (1/a)·g(n) <= f(n); - de (H1): f(n) <= b·g(n) = c2·g(n); - não negatividade: g(n) >= 0 (hipótese do enunciado, f e g assintoticamente não negativas; alternativamente, de 0 <= f(n) <= b·g(n) com b > 0 já segue g(n) >= 0 para n >= n1), e como c1 > 0, tem-se c1·g(n) >= 0. Encadeando: 0 <= c1·g(n) <= f(n) <= c2·g(n) para todo n >= n0.

Isso é exatamente a definição de f(n) = Theta(g(n)), com testemunhas c1 = 1/a, c2 = b e n0 = max(n1, n2). Logo a implicação é verdadeira.

Leitura conceitual: f = O(g) diz que f não cresce mais rápido que g; g = O(f) diz que g não cresce mais rápido que f; as duas juntas prendem f entre dois múltiplos positivos de g, que é a mesma taxa de crescimento a menos de constantes — Theta. Note também a coerência com o item (b): por (b), aplicado com os papeis trocados (f := g, g := f), g = O(f) implica f = Omega(g) — note que (b) demonstra so essa direcao, e e exatamente a de que precisamos aqui; portanto as hipóteses dizem f = O(g) e f = Omega(g), e Theta é precisamente a conjunção de O com Omega.

Exemplo ilustrativo: f(n) = 3n + 5 e g(n) = n. Vale 3n + 5 <= 8n para n >= 1 (b = 8, n1 = 1) e n <= 1·(3n + 5) para n >= 1 (a = 1, n2 = 1); logo c1 = 1, c2 = 8, n0 = 1, e 0 <= n <= 3n + 5 <= 8n para todo n >= 1 — conferindo em n = 1: 0 <= 1 <= 8 <= 8. Correto.

Questão 3(a) (2,0 pontos)

> Enunciado
> A respeito da análise de algoritmos e propriedades das notações assintóticas, responda **V** para as sentenças que julgar verdadeiras e **F** para as sentenças que julgar falsas. Justifique suas respostas. Respostas sem justificativas não serão consideradas.
> a) ( ) (2.0 pontos) A solução da equação de recorrência $T(n) = 2T(n/7) + n \in O(n^2)$.

DE ONDE VEIO Loureiro, ex. 5–8 — mesma técnica (Teorema Mestre), recorrência nova

Resposta. V (verdadeira) — mas é um limite superior correto e NÃO justo

Sentença: a solução de T(n) = 2T(n/7) + n pertence a O(n ^ 2). VERDADEIRA.

1) Resolução da recorrência pelo Teorema Mestre (forma de Brassard & Bratley). A recorrência tem o formato T(n) = a*T(n/b) + c*n ^ k, com a = 2, b = 7, k = 1 e c = 1 (todos naturais, a >= 1, b >= 2, k >= 0, c > 0). Supomos, como na hipótese do teorema, n potência exata de 7 e T assintoticamente não decrescente. Compara-se a com b ^ k: b ^ k = 7 ^ 1 = 7 e a = 2. Como a = 2 < 7 = b ^ k, estamos no primeiro caso, logo T(n) = Theta(n ^ k) = Theta(n).

Intuição (por que o caso 'a < b ^ k' dá Theta(n)): o custo total é a soma dos níveis da árvore de recursão. O nível i tem 2 ^ i subproblemas de tamanho n/7 ^ i, cada um custando n/7 ^ i, ou seja, o nível i custa (2/7) ^ i * n. Somando: n * SOMA_{i>=0} (2/7) ^ i = n * 1/(1 - 2/7) = n * 7/5 = 1,4n. A série é geométrica de razão 2/7 < 1, portanto converge para uma constante: o custo é dominado pela raiz e o total é Theta(n).

2) De Theta(n) para O(n ^ 2). T(n) = Theta(n) implica em particular T(n) = O(n): existem c0 > 0 e n1 tais que T(n) <= c0*n para todo n >= n1. Como n <= n ^ 2 para todo n >= 1, segue T(n) <= c0*n <= c0*n ^ 2 para todo n >= max(n1, 1). TESTEMUNHAS: c = c0 e n0 = max(n1, 1). Verificação em n = n0: T(n0) <= c0*n0 <= c0*n0 ^ 2, pois n0 >= 1. Logo T(n) pertence a O(n ^ 2) e a sentença é VERDADEIRA.

3) Observação que a questão pede implicitamente: O(n ^ 2) é um limite superior VÁLIDO porém FROUXO (não justo). O limite superior justo é O(n), pois T(n) = Theta(n). Isso não torna a sentença falsa: a notação O é apenas um teto, não uma promessa de ajuste fino — Theta(n) está contido em O(n ^ 2), assim como está contido em O(n ^ 3), O(2 ^ n) etc.

Questão 3(b) (2,0 pontos)

> Enunciado
> A respeito da análise de algoritmos e propriedades das notações assintóticas, responda **V** para as sentenças que julgar verdadeiras e **F** para as sentenças que julgar falsas. Justifique suas respostas. Respostas sem justificativas não serão consideradas.
> b) ( ) (2.0 pontos) Se $f(n) = 3 * n^2 - n + 4$ então $f(n) \in \Theta(n^2)$.

DE ONDE VEIO Lista 1, Q3(c) — mesma técnica, polinômio trocado

Resposta. V (verdadeira)

Sentença: f(n) = 3n ^ 2 - n + 4 pertence a Theta(n ^ 2). VERDADEIRA.

Pela definição, precisamos exibir c1 > 0, c2 > 0 e n0 tais que 0 <= c1*n ^ 2 <= 3n ^ 2 - n + 4 <= c2*n ^ 2 para todo n >= n0.

TESTEMUNHAS: c1 = 2, c2 = 6, n0 = 1.

(i) Limite inferior: 2n ^ 2 <= 3n ^ 2 - n + 4. Equivale a 0 <= 3n ^ 2 - n + 4 - 2n ^ 2 = n ^ 2 - n + 4. O polinômio n ^ 2 - n + 4 tem discriminante D = (-1) ^ 2 - 4*1*4 = 1 - 16 = -15 < 0 e coeficiente líder 1 > 0, logo não possui raízes reais e é estritamente positivo para TODO n real. Em particular vale para todo n >= 1. Conferindo em n = n0 = 1: 2*1 ^ 2 = 2 e f(1) = 3 - 1 + 4 = 6; de fato 2 <= 6. OK.

(ii) Limite superior: 3n ^ 2 - n + 4 <= 6n ^ 2. Equivale a 0 <= 6n ^ 2 - 3n ^ 2 + n - 4 = 3n ^ 2 + n - 4 = (3n + 4)(n - 1). Para n >= 1 tem-se 3n + 4 > 0 e n - 1 >= 0, logo o produto é >= 0. Vale para todo n >= 1. Conferindo em n = n0 = 1: f(1) = 6 e 6*1 ^ 2 = 6; de fato 6 <= 6 (igualdade, e a definição usa <=). OK.

(iii) Positividade: para n >= 1, f(n) >= 2n ^ 2 > 0, então as três quantidades da definição são não negativas.

Como as duas desigualdades valem simultaneamente a partir de n0 = 1, concluímos f(n) = Theta(n ^ 2).

CUIDADO COM O FENCEPOST: a escolha c2 = 4 com n0 = 1 seria ERRADA, pois em n = 1 teríamos f(1) = 6 e 4*1 ^ 2 = 4, e 6 <= 4 é falso. Com c2 = 4 seria necessário n0 = 2 (f(2) = 12 - 2 + 4 = 14 <= 4*4 = 16, e 4n ^ 2 - 3n ^ 2 + n - 4 = n ^ 2 + n - 4 >= 0 para n >= 2). Por isso conferimos a desigualdade exatamente em n = n0.

Comentário conceitual: o resultado é o caso particular do fato geral de que todo polinômio de grau d com coeficiente líder positivo é Theta(n ^ d) — os termos de menor ordem (-n e +4) e a constante multiplicativa 3 são exatamente o que a notação assintótica descarta.

Questão 3(c) (2,0 pontos)

> Enunciado
> A respeito da análise de algoritmos e propriedades das notações assintóticas, responda **V** para as sentenças que julgar verdadeiras e **F** para as sentenças que julgar falsas. Justifique suas respostas. Respostas sem justificativas não serão consideradas.
> c) ( ) (2.0 pontos) Sejam as funções $f(n)$ e $g(n)$ assintoticamente positivas e considere que $g(n)$ domina assintoticamente $f(n)$, então $f(n) + g(n) \in \Omega(f(n))$.

DE ONDE VEIO definição de “domina assintoticamente” vem dos slides do Loureiro

Resposta. V (verdadeira)

Sentença: f e g assintoticamente positivas, g domina assintoticamente f; então f(n) + g(n) pertence a Omega(f(n)). VERDADEIRA.

Por definição, f e g assintoticamente positivas significa que existem N1 e N2 com f(n) > 0 para n >= N1 e g(n) > 0 para n >= N2. Tome N = max(N1, N2).

Para todo n >= N vale g(n) > 0, logo f(n) + g(n) >= f(n) + 0 = f(n) = 1 * f(n), e além disso f(n) + g(n) > 0, de modo que ambos os lados são positivos, como exige a definição.

TESTEMUNHAS: c = 1 e n0 = N. Conferindo exatamente em n = n0 = N: f(N) + g(N) >= f(N) porque g(N) > 0. OK.

Portanto existe c = 1 > 0 e existe n0 = N tais que, para todo n >= n0, f(n) + g(n) >= c*f(n), ou seja, f(n) + g(n) = Omega(f(n)).

OBSERVAÇÃO IMPORTANTE: a hipótese de que g DOMINA assintoticamente f (nos slides do Loureiro: existem constantes positivas C e m tais que |f(n)| <= C*|g(n)| para todo n >= m — isto é, f(n) = O(g(n)), e NÃO f(n) = o(g(n)); a letra C, maiúscula, para não colidir com a testemunha c = 1 acima) NÃO foi utilizada e NÃO é necessária. A conclusão vale para quaisquer f e g assintoticamente não negativas — basta que g não seja negativa a partir de certo ponto para que somar g só possa aumentar o valor. A dominância é, aqui, dado supérfluo do enunciado (as instruções da prova alertam que pode haver informação supérflua). O que a dominância acrescenta, se quisermos usá-la, é o resultado MAIS FORTE f(n) + g(n) = Theta(g(n)): como f(n) <= C*g(n) para n >= m (a constante C da definição, que pode ser maior que 1), tem-se, para todo n >= max(N, m) — faixa em que f e g são positivas, de modo que o módulo pode ser retirado —, g(n) <= f(n) + g(n) <= (1 + C)*g(n).

Esta sentença é exatamente a mesma da Questão 4(b), e a resposta lá é a mesma: sim, pode-se afirmar.

Questão 4(a) (2,0 pontos)

> Enunciado
> A respeito da análise de algoritmos e propriedades das notações assintóticas, responda:
> (a) (2 pontos) Alice e Bob estão discutindo sobre o desempenho de seus algoritmos de ordenação. Alice afirma que o seu algoritmo de ordenação executando em tempo $\theta(n * \lg n)$ é melhor que algoritmo de Bob que executa em tempo $\theta(n^2)$. Para decidir qual é melhor algoritmo, implementaram e executaram os dois algoritmos no mesmo ambiente (mesma arquitetura, mesmo sistema operacional, mesma linguagem de programação e mesmo compilador) em uma série de dados. Para espanto de Alice, descobriram que quando $n < 100$ o algoritmo $\theta(n^2)$ é mais rápido e apenas quando $n \geq 100$ é que o algoritmo $\theta(n * \lg n)$ é melhor. Explique como essa situação pode ocorrer.

DE ONDE VEIO Lista 3, Q1 — **idêntica, exceto por uma expressão**: a lista diz "executaram *ambos os* algoritmos", a prova diz "executaram *os dois* algoritmos"

Resposta. É perfeitamente possível: Theta descarta constantes multiplicativas e termos de menor ordem, e só garante o comportamento a partir de um n0; para n pequeno as constantes mandam.

A situação descrita não contradiz coisa alguma. A notação Theta é uma afirmação sobre o COMPORTAMENTO ASSINTÓTICO, isto é, sobre o que acontece quando n cresce indefinidamente, e ela deliberadamente joga fora duas coisas: (i) as CONSTANTES MULTIPLICATIVAS e (ii) os TERMOS DE MENOR ORDEM. Além disso, a própria definição só exige que as desigualdades valham 'para todo n >= n0' — nada é prometido para os n pequenos.

Par concreto que reproduz exatamente o relato de Alice e Bob: Alice: T_A(n) = 15*n*lg n + 20 (de fato Theta(n lg n)) Bob: T_B(n) = n ^ 2 (de fato Theta(n ^ 2))

Valores: n = 1: T_A = 20 contra T_B = 1 -> Bob mais rápido n = 50: T_A ~ 4.253 contra T_B = 2.500 -> Bob mais rápido n = 99: T_A ~ 9.865 contra T_B = 9.801 -> Bob mais rápido n = 100: T_A ~ 9.986 contra T_B = 10.000 -> ALICE mais rápida n = 1000: T_A ~ 149.507 contra T_B = 1.000.000 -> Alice mais rápida, e por muito

O termo constante +20 é ESSENCIAL no exemplo: sem ele teríamos T_A(1) = 15*1*lg 1 = 0, pois lg 1 = 0, e Alice 'venceria' em n = 1 com tempo zero, o que contradiria o enunciado. Nenhum algoritmo real custa zero: sempre há um custo fixo de inicialização, e é ele que o termo constante modela.

PONTO DE CRUZAMENTO. Igualando as duas funções (na forma dominante) 15*n*lg n = n ^ 2 e dividindo por n*lg n — divisão legítima apenas para n >= 2, pois exige lg n > 0 — obtemos n / lg n = 15. A função n/lg n tem mínimo em n = e e é monótona crescente a partir daí; restringindo-nos a n >= 3, ela é crescente e portanto a raiz é única. Numericamente, 99 / lg 99 = 99 / 6,6294 ~ 14,93 (< 15) 100 / lg 100 = 100 / 6,6439 ~ 15,05 (> 15) ou seja, a raiz de n / lg n = 15 está em n ~ 99,56, ENTRE 99 e 100 — não 'exatamente em 100'. (Com o termo +20 incluído, isto é, resolvendo 15*n*lg n + 20 = n ^ 2, o cruzamento exato das duas funções do exemplo fica em n ~ 99,82 — também entre 99 e 100.) Como n é inteiro, o primeiro inteiro em que Alice passa a vencer é n = 100, exatamente o que Alice e Bob observaram.

CUIDADO CONCEITUAL: esse 100 é o PONTO DE CRUZAMENTO entre T_A e T_B, provocado pela constante multiplicativa 15 de Alice; ele NÃO é o n0 das definições de Theta de cada algoritmo (para T_A e T_B as testemunhas de Theta podem ser obtidas já com n0 = 2). É um erro chamar o cruzamento de 'n0 do par'.

O que mais pode explicar a diferença na prática, além da constante multiplicativa: - TERMOS DE MENOR ORDEM: o algoritmo de Alice pode custar, digamos, 15n lg n + 40n + 20; para n pequeno os termos 40n e 20 pesam mais que o termo dominante. - FATORES QUE O MODELO RAM NÃO ENXERGA: localidade de cache (um algoritmo simples e sequencial percorre memória contígua e sofre menos cache miss), custo de chamada de função e de recursão (empilhar/desempilhar quadros), alocação de memória auxiliar e cópia de vetores.

ESTE É EXATAMENTE O CASO CLÁSSICO insertion sort vs. mergesort: insertion sort é Theta(n ^ 2) mas com constante minúscula e ótimo comportamento de cache, sendo mais rápido para entradas pequenas; mergesort é Theta(n lg n) mas paga recursão e vetor auxiliar. Por isso implementações reais de mergesort/quicksort trocam para insertion sort quando o subvetor fica menor que um limiar (tipicamente entre 10 e 50 elementos).

CONCLUSÃO: Alice está certa assintoticamente — para n suficientemente grande seu algoritmo é melhor, e a vantagem cresce sem limite. Mas 'assintoticamente melhor' não significa 'melhor para todo n'. Se a aplicação só processa entradas com n < 100, o algoritmo de Bob é a escolha correta.

Questão 4(b) (2,0 pontos)

> Enunciado
> A respeito da análise de algoritmos e propriedades das notações assintóticas, responda:
> (b) (2 pontos) Sejam as funções $f(n)$ e $g(n)$ assintoticamente positivas e considere que $g(n)$ domina assintoticamente $f(n)$. Pode-se afirmar que $f(n) + g(n) \in \Omega(f(n))$? Justifique formalmente sua resposta.

DE ONDE VEIO repete a Questão 3(c) desta mesma prova

Resposta. Sim, pode-se afirmar: f(n) + g(n) pertence a Omega(f(n))

Sim. E a demonstração é a mesma da Questão 3(c) — as duas questões são a MESMA sentença, e por isso as respostas são idênticas e consistentes.

Hipóteses: f e g são assintoticamente positivas, ou seja, existem N1 e N2 tais que f(n) > 0 para todo n >= N1 e g(n) > 0 para todo n >= N2. Defina N = max(N1, N2).

Demonstração formal. Precisamos exibir c > 0 e n0 tais que 0 <= c*f(n) <= f(n) + g(n) para todo n >= n0. Tome c = 1 e n0 = N. Para todo n >= n0 = N: - f(n) > 0, logo c*f(n) = f(n) > 0 (a condição de não negatividade está satisfeita); - g(n) > 0, portanto f(n) + g(n) >= f(n) + 0 = f(n) = c*f(n). Conferência exatamente em n = n0 = N: f(N) + g(N) >= f(N), pois g(N) > 0. A desigualdade vale já no primeiro ponto, sem erro de fencepost.

Como exibimos testemunhas concretas c = 1 e n0 = N, satisfazendo a definição 'existe c > 0, existe n0, para todo n >= n0', concluímos f(n) + g(n) = Omega(f(n)).

OBSERVAÇÃO QUE VALE PONTOS: a hipótese de que g DOMINA assintoticamente f (nos slides do Loureiro: |f(n)| <= C*|g(n)| para todo n >= m, com C e m constantes positivas — ou seja, f(n) = O(g(n)); C maiúscula para não colidir com a testemunha c = 1) NÃO foi usada em momento algum e é desnecessária. O argumento só precisa que g seja não negativa a partir de certo ponto — somar uma parcela não negativa nunca diminui o valor. Logo a afirmação vale para QUAISQUER f e g assintoticamente não negativas; a dominância é informação supérflua do enunciado.

Se quisermos usar a dominância, ela permite uma conclusão mais forte: como a dominância dá f(n) <= C*g(n) para todo n >= m, temos, para n >= max(N, m), 1*g(n) <= f(n) + g(n) <= (1 + C)*g(n), ou seja, f(n) + g(n) = Theta(g(n)) — a soma é governada pela função dominante (regra da soma). Isso, aliás, também dá Omega(f(n)) por outro caminho, já que f(n) + g(n) >= g(n) >= (1/C)*f(n) nessa faixa.

Questão 5(a) (3,0 pontos)

Enunciado

Suponha que, para resolver um problema com entradas de tamanho n, você tenha que escolher um dentre dois algoritmos A e B.
1. O algoritmo A resolve problemas dividindo-o em um único subproblema de tamanho n/5, resolve recursivamente o subproblema e então combina as soluções em tempo θ(n).
2. O algoritmo B resolve problemas de tamanho n, dividindo-o em único subproblema de tamanho n − 1, resolve recursivamente o subproblema e então combina as soluções em tempo constante.
Considere essas informações e responda:
(a) (3.0 pontos) Qual o consumo de tempo de cada um dos algoritmos? Expresse sua resposta em termos da notação θ. Justifique sua resposta. Respostas sem justificativas não serão consideradas.

DE ONDE VEIO Lista 3, Q4(d) e (e) — mesma técnica de recorrência, casos novos (antes da atualização de 09/09/2026 era a Q3)

Resposta. Algoritmo A: Theta(n) (n potência exata de 5, como supõe o formulário). Algoritmo B: Theta(n).

ALGORITMO A. Ele gera UM ÚNICO subproblema de tamanho n/5 e gasta Theta(n) para dividir/combinar. A recorrência é T_A(n) = T_A(n/5) + Theta(n), com T_A(1) = Theta(1).

Aplicando o Teorema Mestre na forma de Brassard & Bratley (T(n) = a*T(n/b) + c*n^k): a = 1 (um único subproblema), b = 5, k = 1, c > 0. b^k = 5^1 = 5. Como a = 1 < 5 = b^k, cai-se no primeiro caso e T_A(n) = Theta(n^k) = Theta(n).

HIPÓTESE DO FORMULÁRIO. O teorema impresso na prova exige f(n) = c*n^k EXATAMENTE, e o enunciado só diz que combinar custa Theta(n). SUPOSIÇÃO (declarada, como a instrução 2 da prova pede quando o enunciado é omisso): o custo de combinar é positivo em todo n >= 1 — toda combinação executa ao menos uma operação. A ponte: como esse custo é Theta(n) e positivo, existem constantes 0 < c1 <= c2 com c1*n <= f(n) <= c2*n para todo n >= 1 (os finitos n abaixo do n0 do Theta são absorvidos ajustando c1 e c2). Defina T_inf(n) = T_inf(n/5) + c1*n e T_sup(n) = T_sup(n/5) + c2*n, com o mesmo caso base T_A(1). Por indução sobre n = 5^j, T_inf(n) <= T_A(n) <= T_sup(n): na base as três são iguais, e no passo T_A(n) = T_A(n/5) + f(n) fica entre T_inf(n/5) + c1*n e T_sup(n/5) + c2*n. As duas recorrências extremas já têm a forma exata do formulário (a = 1, b = 5, k = 1, c = c1 ou c2), com n potência exata de 5, e expandindo dão T_sup(n) = T_A(1) + c2*(5n - 5)/4, que é não decrescente; o mesmo para T_inf. Logo ambas são Theta(n), e T_A, presa entre elas, também é — para n potência exata de 5, que é exatamente o domínio que o formulário supõe. Testemunhas explícitas, das formas fechadas: para n potência de 5 com n >= 5, T_A(n) >= T_inf(n) = T_A(1) + c1*(5n - 5)/4 >= c1*n (pois 5n - 5 >= 4n quando n >= 5), e T_A(n) <= T_sup(n) = T_A(1) + c2*(5n - 5)/4 <= (T_A(1) + 5*c2/4)*n. Logo c1' = c1, c2' = T_A(1) + 5*c2/4 e n0 = 5 (em n = 5: T_inf(5) = T_A(1) + 5*c1 >= 5*c1, e T_sup(5) = T_A(1) + 5*c2 <= 5*T_A(1) + 25*c2/4).

Intuição / verificação pela árvore de recursão: a árvore é um caminho (um filho por nível). O nível i custa c*(n/5^i). Somando a série geométrica de razão 1/5: c*n * SOMA_{i>=0} (1/5)^i = c*n * 1/(1 - 1/5) = c*n * 5/4 = 1,25*c*n, que é uma COTA SUPERIOR: a árvore é finita, e a soma exata dos níveis é c*(5n - 5)/4, mais o caso base. É uma série geométrica DECRESCENTE: o termo do topo, c*n, domina: para n potência de 5 com n >= 5, a soma dos níveis fica entre c*n e no máximo 5/4 disso, fora o caso base (em n = 1 não há nível de combinação, e a soma é 0). Daí T_A(n) = Theta(n). A profundidade da recursão é log_5 n = Theta(log n).

ATENÇÃO À PEGADINHA: é tentador achar que 'dividir em subproblemas' produz algo logarítmico, mas aqui o custo de combinação é linear e é ele que domina; a redução de tamanho por fator 5 só faz a série convergir.

ALGORITMO B. Ele gera UM ÚNICO subproblema de tamanho n-1 e combina em tempo constante: T_B(n) = T_B(n-1) + Theta(1), com T_B(1) = Theta(1).

```
O TEOREMA MESTRE NÃO SE APLICA aqui: a recorrência não tem o formato a*T(n/b) + f(n) — o tamanho do subproblema é
obtido por SUBTRAÇÃO (n-1) e não por DIVISÃO (n/b). Resolve-se por expansão (método iterativo). Escrevendo o custo
de combinação como uma constante c > 0:
  T_B(n) = T_B(n-1) + c
         = T_B(n-2) + 2c
         = T_B(n-3) + 3c
         = ...
         = T_B(n-i) + i*c.
Parando em n - i = 1, isto é, i = n - 1:
  T_B(n) = T_B(1) + (n-1)*c = c*n + (T_B(1) - c).
Em forma fechada, o somatório é SOMA_{i=1}^{n-1} c = c*(n-1), mais o caso base.
```

Portanto T_B(n) = c*n + d, uma função linear em n, e assim T_B(n) = Theta(n). Testemunhas (supondo T_B(1) = c, ou seja, T_B(n) = c*n): c1 = c, c2 = c, n0 = 1, pois c*n <= c*n <= c*n para todo n >= 1. No caso geral T_B(n) = c*n + d, o sinal de d = T_B(1) - c é qualquer (o caso base pode custar menos que uma combinação), mas c + d = T_B(1) > 0. Escreva T_B(n)/n = c*(n-1)/n + T_B(1)/n: é uma média ponderada de c e T_B(1), com pesos (n-1)/n e 1/n. Logo, para todo n >= 1, min(c, T_B(1))*n <= T_B(n) <= max(c, T_B(1))*n, e as testemunhas são c1 = min(c, T_B(1)), c2 = max(c, T_B(1)), n0 = 1 (conferindo em n = 1: T_B(1) está entre min e max de {c, T_B(1)}, correto). Se o custo de combinar variar com n entre duas constantes 0 < c' <= c'' (é o que Theta(1) garante a partir de algum n0; os finitos n abaixo dele são absorvidos como em A, pela mesma SUPOSIÇÃO de positividade, que vale também para o custo constante de combinar de B), o mesmo argumento, feito com c' e com c'', prende T_B entre duas funções lineares.

CONCLUSÃO DE (a): T_A(n) = Theta(n), para n potência exata de 5 (o domínio do formulário), e T_B(n) = Theta(n), para todo n >= 1.

Questão 5(b) (1,0 pontos)

Enunciado

Suponha que, para resolver um problema com entradas de tamanho n, você tenha que escolher um dentre dois algoritmos A e B.
1. O algoritmo A resolve problemas dividindo-o em um único subproblema de tamanho n/5, resolve recursivamente o subproblema e então combina as soluções em tempo $\theta(n)$.
2. O algoritmo B resolve problemas de tamanho n, dividindo-o em único subproblema de tamanho $n - 1$, resolve recursivamente o subproblema e então combina as soluções em tempo constante.
Considere essas informações e responda:
(b) (1.0 ponto) Qual dos algoritmos é assintoticamente mais eficiente no pior caso? Justifique sua resposta. Respostas sem justificativas não serão consideradas.

Resposta. Nenhum dos dois — são assintoticamente equivalentes, ambos Theta(n).

Como mostrado em (a), T_A(n) = Theta(n) e T_B(n) = Theta(n). Logo T_A(n) = Theta(T_B(n)) e T_B(n) = Theta(T_A(n)): as duas funções de custo pertencem à MESMA classe assintótica. Portanto NENHUM dos dois algoritmos é assintoticamente mais eficiente que o outro no pior caso — eles são assintoticamente EQUIVALENTES.

Esta é a pegadinha da questão: a aparência dos dois algoritmos é bem diferente (um reduz o tamanho por um fator 5 com combinação linear, o outro reduz de 1 em 1 com combinação constante), mas os dois efeitos se compensam exatamente. Em A a recursão é rasa (log_5 n níveis) e cada nível é caro (linear, em série geométrica que converge para 5n/4); em B a recursão é profunda (n níveis) e cada nível é barato (constante, somando c*n). Responder 'A é melhor' ou 'B é melhor' é errado.

Onde a diferença realmente aparece — e não é assintótica no tempo: 1) CONSTANTES MULTIPLICATIVAS: A gasta cerca de (5/4)*c_A*n, onde c_A é o custo POR ELEMENTO da combinação linear de A; B gasta cerca de c_B*n, onde c_B é o custo de UMA combinação de B. São constantes de naturezas diferentes, que dependem da implementação — o enunciado não permite dizer qual algoritmo tem a menor. E a notação Theta descarta constantes, então isso não muda a resposta assintótica. 2) CONSUMO DE PILHA (memória), e aqui sim há diferença ASSINTÓTICA: A faz Theta(log n) chamadas recursivas aninhadas, enquanto B faz Theta(n). Em uma implementação recursiva sem otimização de chamada de cauda, B usa espaço de pilha Theta(n) e pode causar estouro de pilha para n grande, ao passo que A usa Theta(log n) e é seguro. Na prática, portanto, A tende a ser preferível — mas por causa do espaço, não do tempo.

RESPOSTA FINAL: nenhum é assintoticamente mais eficiente em tempo no pior caso; ambos são Theta(n). Se for preciso desempatar, o critério é o espaço de pilha, e aí A (Theta(log n)) leva vantagem sobre B (Theta(n)).

Informações que podem ser úteis

Anexo da prova (transcrição literal)

Somatórios e Logaritmos:

- $\Sigma_{i=1}^{n} (i) = n(n + 1)/2$
- $\Sigma_{i=1}^{n} (a^i) = a(a^n - 1)/(a - 1)$
- $\log (a * b) = \log a + \log b$
- $\log (a/b) = \log a - \log b$
- $\log_b a = \log_c a / \log_c b$
- $\log_b a^c = c * \log_b a$
- $\lg a = \log_2 a$

Hierarquia de funções:

A seguinte hierarquia de funções pode ser definida do ponto de vista assintótico: $1 < \log(n) < \sqrt{n} < n < n * \log(n) < n^c < c^n < n! < n^n$ onde c é uma constante arbitrária inteira ≥ 2.

Teorema mestre:

Sejam as constantes naturais $a \geq 1$, $b \geq 2$ e $k \geq 0$. Considere a função f(n) da forma $c*n^k$, na qual $c > 0$ e $\in R$, seja a equação de recorrência dada por $T(n) = aT(n/b)+f(n)$ e suponha n uma potência exata de b. Considere ainda que T é uma função que leva números naturais em números reais positivos e que T é assintoticamente não decrescente, Nessas condições, [BRASSARD and BRATLEY, 1996] indicam que a solução dessa recorrência é limitada assintoticamente por

$$T(n) = \theta(n^k) \text{ se } a < b^k; \quad \theta(n^k * \lg n) \text{ se } a = b^k; \quad \theta(n^{\log_b a}) \text{ se } a > b^k$$

Referências

[BRASSARD and BRATLEY, 1996] BRASSARD, G. and BRATLEY, P. (1996). Fundamentals of Algorithms. Prentice-Hall.

Sobre esta resolução. Produzida com verificação adversarial em contexto isolado e conferida numericamente: o algoritmo do campo minado foi simulado (a razão máxima passos/m observada em m = 1…299 foi 12,888, confirmando a testemunha c = 14), as recorrências da Questão 5 foram resolvidas por expansão direta, e as testemunhas da Questão 3(b) foram testadas em 300 000 valores de n. Os scripts estão em `RESOLUCOES/verificacao/`.
Auditoria de 06/10/2026. Enunciados, cabeçalho, instruções e anexo transcritos literalmente da prova (antes estavam resumidos). Corrigidos, sem mudar nenhuma resposta: a definição de “domina assintoticamente” nas 3(c) e 4(b) (nos slides do Loureiro é $|f| \leq C|g|$, um O, e não um o); o ponto de cruzamento da

4(a) (a raiz de $n/\lg n = 15$ é 99,56; 99,82 é a do exemplo com +20); as testemunhas de B na 5(a) (caso base mais barato que uma combinação); a ponte entre $\Theta(n)$ e a forma $c \cdot n^k$ do formulário, com testemunhas numéricas, na 5(a); e a comparação de constantes da 5(b). Patches no lote 7 e 8 de `geradores/correcoes.py`. Verificação: 4 rodadas de 4 verificadores independentes; todas deram HOLDS em todas as questões, e os achados de cada rodada (cada vez menores) foram corrigidos e reverificados. IFMG Campus Formiga.